Informe funcional del dashboard de AgroBot

Análisis de todas las funcionalidades de la interfaz de operación · versión del 27 de octubre de 2026 · código: jetson/dashboard/index.html y jetson/backend/jetson_server.py

1. Resumen

El dashboard es la única interfaz de operación del robot: una aplicación web de una sola página, servida por el propio backend de la Jetson y sin dependencias externas (funciona en la red aislada del punto de acceso, sin internet). Tiene 11 pestañas más 4 páginas auxiliares, 93 botones, 53 deslizadores, 10 interruptores y 5 lienzos de dibujo en 4 259 líneas (240 KB). Se comunica con el backend por WebSocket (tiempo real: 25 tipos de mensaje manejados) y por una API REST de 128 rutas, de las cuales 72 las usan las páginas del dashboard.

Este informe describe, pestaña por pestaña, cada control (marcado con un número rojo en una captura real del dashboard con el robot conectado), qué hace y con qué ruta, mensaje y módulo del backend se conecta; recoge la arquitectura, los canales de comunicación, los mecanismos de seguridad y robustez, y una evaluación crítica con los problemas encontrados y corregidos durante el análisis.

2. Alcance y método

- Análisis estático de index.html (estructura, manejadores de eventos, llamadas fetch y mensajes WebSocket) y de las rutas registradas en jetson_server.py, recoleccion.py y etiquetado.py (tools/inventario_dashboard.py).
- Capturas reales de las 11 pestañas con el backend y el robot conectados, a 1500 px de ancho, mediante Chromium sin cabeza controlado por el protocolo CDP; la posición de cada control se mide en el navegador y se dibujan cuadros y números rojos (tools/capturas_anotadas_dashboard.py).
- Descripciones de cada control contrastadas con los textos de ayuda de la propia interfaz, los docstrings de los manejadores y el código; la especificación es única (tools/dashboard_spec.py).
- Límites: no se midió el rendimiento ni se probaron todos los botones con el robot en movimiento; los controles que mueven el robot (misiones, maniobras, calibraciones) se describen por el código, no por una prueba funcional de cada uno.

3. Arquitectura

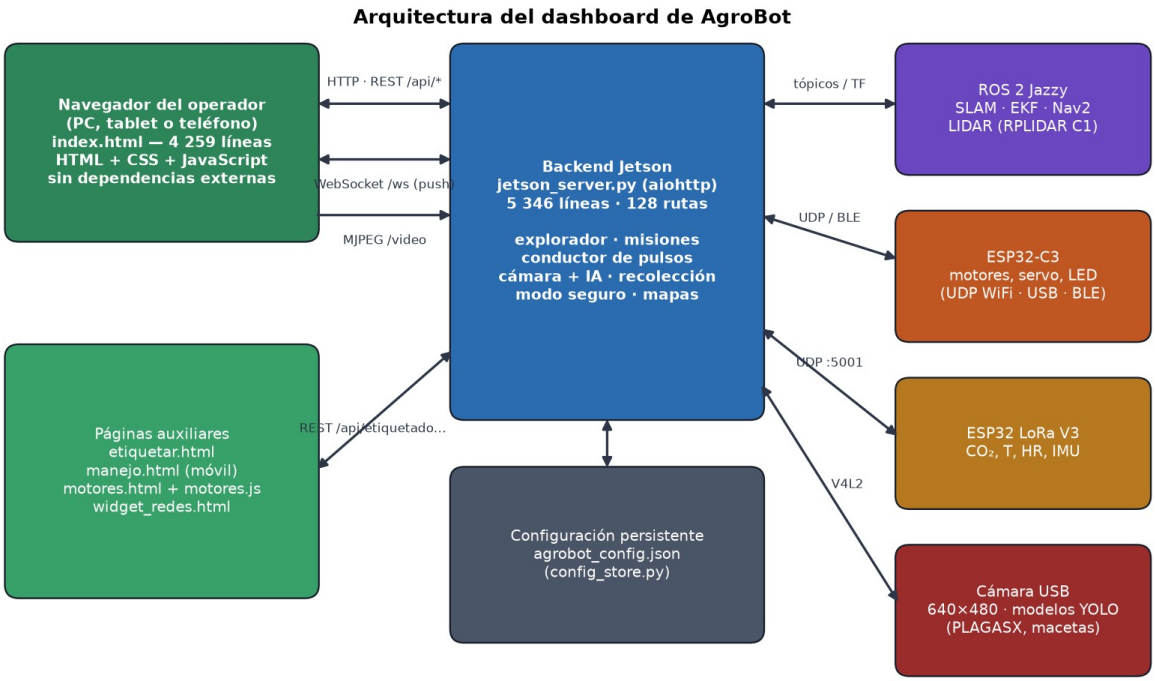


Figura 1. Arquitectura del dashboard y de lo que controla.

El navegador descarga index.html (HTML, CSS y JavaScript en un solo archivo, sin librerías ni fuentes externas) y abre un WebSocket con el backend. Por ahí llegan, sin que el navegador pregunte, la pose, el mapa, el scan, las detecciones y la telemetría; las órdenes del operador salen por peticiones REST (HTTP), a propósito: en el teléfono el WebSocket se cae al bloquear la pantalla y una petición HTTP o llega o devuelve un error. La excepción es el STOP, que va por las dos vías a la vez.

Indicador	Valor
Líneas / tamaño de index.html	4259 / 240 KB
Pestañas + páginas auxiliares	11 + 4
Botones / deslizadores / interruptores / lienzos	93 / 53 / 10 / 5
Elementos con identificador	343
Llamadas fetch / sondeos periódicos (setInterval)	64 / 8
Rutas REST registradas / usadas por alguna página	128 / 72
Mensajes WebSocket manejados / emitidos por el backend	25 / 32
Consultas de ancho (@media) / atributos title / aria-label	16 / 32 / 6
Diálogos de confirmación (confirm) / usos de localStorage	11 / 16

4. Funcionalidades por pestaña

En cada captura, los números rojos remiten a la tabla que la acompaña. Las tablas indican la función, el canal con el backend (ruta REST o mensaje WebSocket) y el módulo del backend que responde.

4.1 Dashboard (vista principal)

Es la pantalla de operación. Reúne en una sola vista la telemetría ambiental, el estado del LIDAR, el video con las detecciones, el minimapa y el radar, y el control manual.

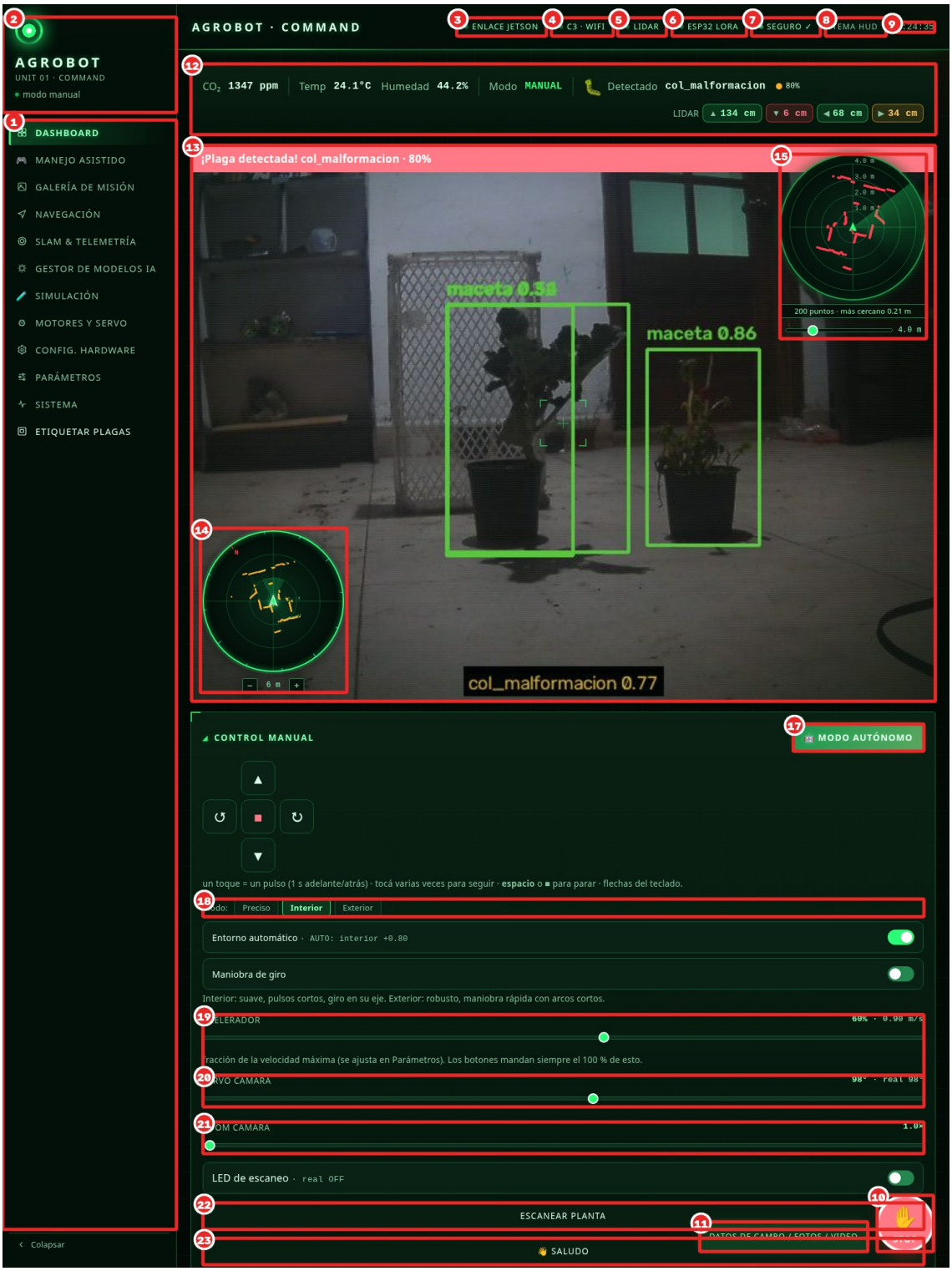


Figura 2. Dashboard (vista principal). Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Lista de pestañas	Navega entre las 11 pestañas y la página aparte «Etiquetar plagas». Tiene scroll con deslizador visible para pantallas bajas.	Local (JavaScript)	—
2	Identidad y estado	Logotipo animado (reactor) y modo de manejo actual («modo manual»).	WS drive_mode	jetson_server.py
3	Indicador «enlace Jetson»	Verde cuando el WebSocket con el backend está abierto; se reconecta solo.	WebSocket /ws	jetson_server.py
4	Indicador del actuador (C3)	Estado del enlace con el ESP32-C3 y la vía en uso (WiFi, USB o BLE).	WS actuator_status	enlace_c3.py, udp_actuator_link.py
5	Indicador del LIDAR	Verde si llegan scans frescos; ámbar/rojo si el dato está viejo o no llega.	WS health	jetson_server.py (_salud_completa)
6	Indicador del ESP32 LoRa	Frescura de la telemetría ambiental y del IMU.	WS health, env_telemetry	lora_sensor_ble.py
7	Indicador de seguridad	Muestra el nivel («normal», «precaución», «seguro»); al pulsarlo permite rearmar el modo seguro.	WS seguridad; POST /api/seguridad/rearmar	modo_seguro.py
8	Tema HUD / SOL	Alterna entre el tema HUD verde y el tema claro para leer a pleno sol; se recuerda por navegador.	Local (localStorage)	—
9	Reloj	Hora del navegador.	Local	—
10	Botón STOP	Parada de emergencia: detiene misión, exploración, patrulla, Nav2, pulsos y motores. Se envía por WebSocket y por HTTP a la vez (redundancia) y reacciona al primer toque (pointerdown).	WS emergency_stop + POST /api/emergencia	jetson_server.py (_parada_total)
11	Datos de campo / Fotos / Video	Abre el panel de recolección: iniciar/detener, foto, video, muestras guardadas y espacio libre en disco.	GET/POST /api/recoleccion[/start|stop|photo|video]	recoleccion.py, grabador.py
12	Barra de estado	CO_2, temperatura y humedad del ESP32 LoRa (marcadas como simuladas si el SCD40 no responde), modo de operación, última detección con su confianza y huecos libres del LIDAR por lado (verde/ámbar/rojo).	WS env_telemetry, detection, lidar_distancias	lora_sensor_codec.py, camera.py, lidar_reactive
13	Video en vivo	MJPEG de la cámara con las cajas de detección dibujadas por el backend; las cajas de plaga de imagen completa se anotan como rótulo. Banner rojo «¡Plaga detectada!» para las clases de plaga (no para macetas). Muestra FPS.	GET /video; WS detection	camera.py
14	Minimapa	Mapa del SLAM centrado en el robot, «rumbo arriba» o «norte arriba» con un clic; muestra el scan vivo, el cono de la cámara, la patrulla y las plantas. Botones −/+ cambian el alcance (3, 6, 12, 24 m).	WS map, pose, scan_mapa	ros_bridge.py, slam
15	Radar LIDAR + alcance	Vista polar del scan del LIDAR con barrido animado y deslizador de alcance (0,5–16 m).	WS lidar_scan	lidar_reactive
16	Control manual	Cruceta (adelante/atrás/giros/parar) con un pulso por toque, o teclas WASD y flechas; el espacio y ■ paran.	POST /api/drive	pulse_drive.py (ConductorPulsos)
17	Modo autónomo	Inicia o detiene la exploración autónoma del robot.	POST /api/mission/start|stop	explorador.py
18	Modo de manejo y entorno	Preciso (pulsos cortos, la mitad de largos), Interior (suave, giro en su eje) o Exterior (robusto, maniobra rápida con arcos cortos). Debajo, «Entorno automático» propone interior/exterior con la puntuación del clasificador y «Maniobra de giro» activa el giro con arcos.	GET/POST /api/drive/mode, /api/entorno; WS entorno	entorno_auto.py, pulse_drive.py

N.º	Control	Qué hace	Canal con el backend	Módulo
19	Acelerador	Fracción de la velocidad máxima que usan los botones (10–100 %); se recuerda en el navegador.	Local (localStorage)	—
20	Servo de la cámara	Inclina la cámara entre 25° y 160° (reposo 98°); muestra el valor pedido y el real devuelto por el actuador.	WS actuator / actuator_setpoint	udp_actuator_link.py
21	Zoom de la cámara	Zoom digital 1–4× (recorte central a 1280×720); solo con el robot quieto, vuelve a 1× al iniciar la autonomía.	GET/POST /api/camara/zoom	camera.py
22	Escanear planta	Lanza un barrido de la cámara con el servo.	POST /api/patrol/scan	patrol.py
23	Saludo	Secuencia de demostración: giros, mira arriba, foco y barrido.	POST /api/saludo	saludo.py
24	Grabar video	Inicia y detiene la grabación de video a disco.	POST /api/video/record	grabador.py

4.2 Manejo asistido

Junta, sin duplicar, las tarjetas reales del dashboard (estado, cámara, controles, mapa) y añade el aprendizaje de las correcciones del operador sobre la autonomía.

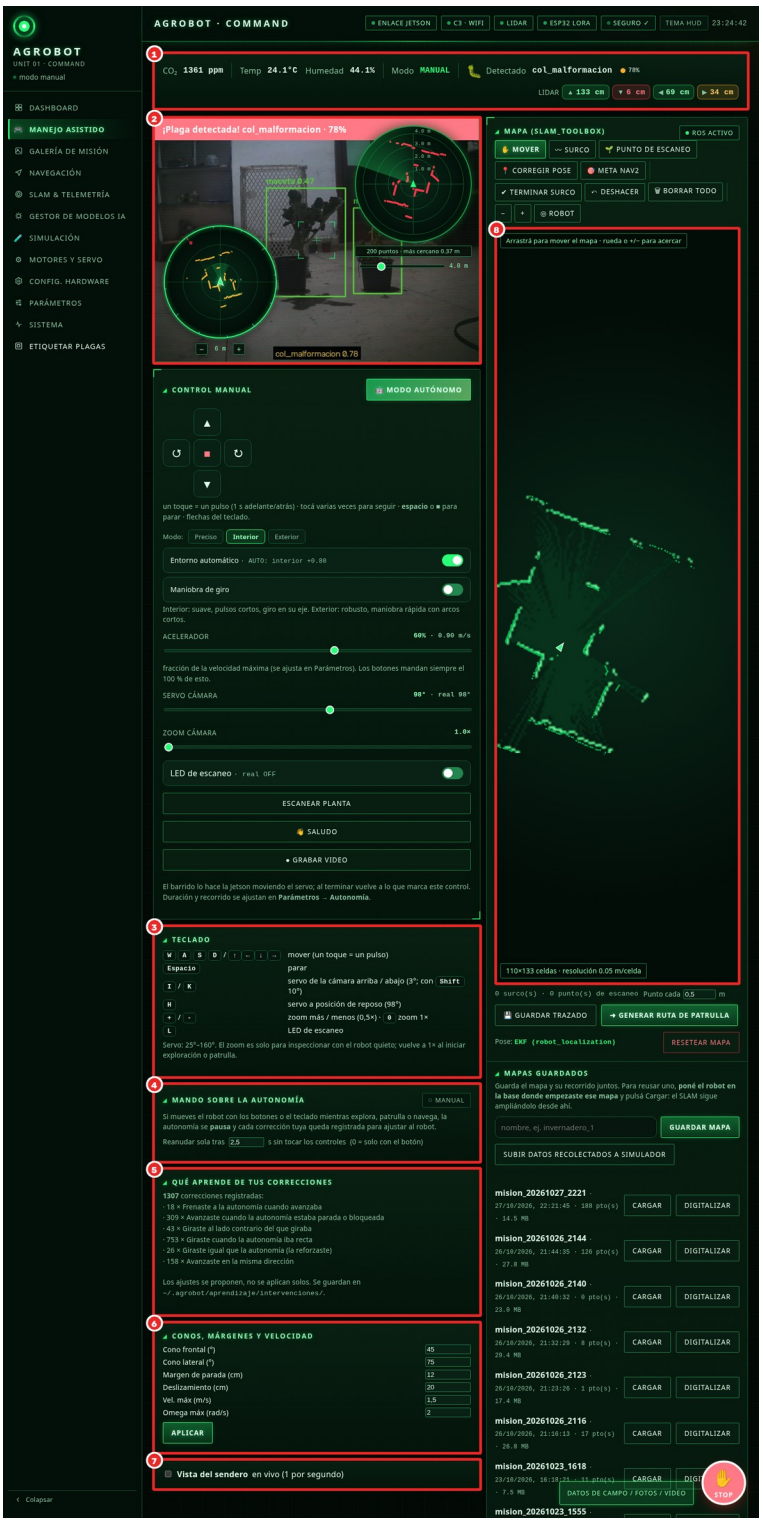


Figura 3. Manejo asistido. Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Estado	La misma barra de estado del Dashboard.	WS (ver Dashboard)	—
2	Cámara con minimapa y radar	La tarjeta de la cámara se «presta» a esta vista, con sus mismos identificadores y manejadores.	GET /video	camera.py
3	Teclado	Tabla de atajos: WASD/flechas, espacio, I/K servo, H reposo, +/− zoom, L LED.	Local	—

N.º	Control	Qué hace	Canal con el backend	Módulo
4	Mando sobre la autonomía	Si mueves el robot mientras explora, patrulla o navega, la autonomía se pausa y la corrección queda registrada. Permite reanudar a mano o sola tras N s sin tocar.	GET /api/asistido; POST /api/asistido/reanudar|soltar, /api/params/autonomia	asistido.py
5	Qué aprende de tus correcciones	Cuenta y clasifica las correcciones (frenaste, giraste al lado contrario…) y propone ajustes que NO se aplican solos.	GET /api/asistido; POST /api/asistido/aplicar	asistido.py
6	Conos, márgenes y velocidad	Cono frontal/lateral, margen de parada, deslizamiento, velocidad y omega máximos; botón APLICAR.	POST /api/params/slam, /api/params/control	config_store.py
7	Vista del sendero	Imagen en vivo (1 por segundo) con lo que el detector de hileras ve y su proyección cenital.	GET /api/sendero/vista.jpg	sendero.py, sendero_apariencia.py
8	Mapa SLAM	Mapa con herramientas de trazado (ver Navegación).	WS map, scan_mapa	ros_bridge.py

4.3 Galería de misión

Colección de las capturas que el robot guarda al detectar una plaga, con el recorte y la caja dibujada.

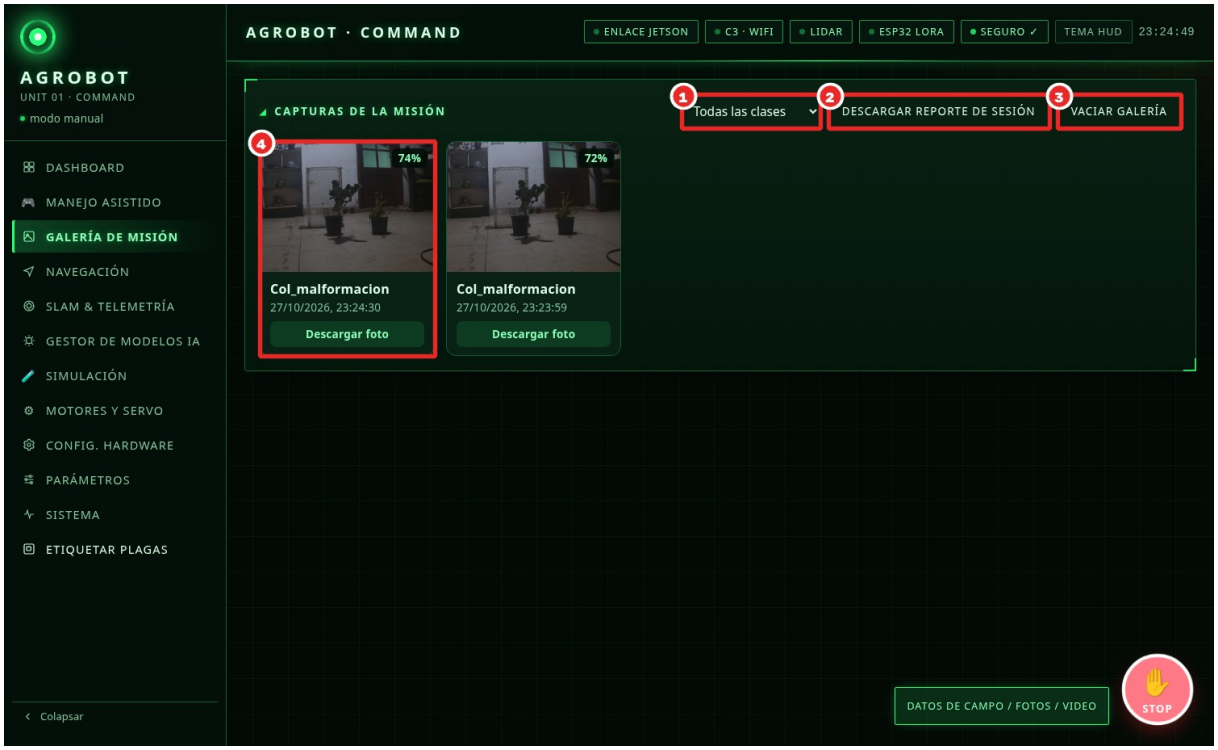


Figura 4. Galería de misión. Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Filtro por clase	Muestra solo las capturas de una clase de plaga.	Local (sobre GET /api/captures)	—
2	Descargar reporte de sesión	Genera y descarga el reporte de la sesión con las capturas y su resumen.	GET /api/session-report	jetson_server.py
3	Vaciar galería	Borra las capturas de la galería (pide confirmación).	POST /api/captures/clear	jetson_server.py
4	Tarjeta de captura	Foto con la confianza, la clase, la fecha y enlace de descarga.	WS capture; GET /captures/<archivo>	camera.py

4.4 Navegación — modo manual (grabar recorrido y mapa)

Tres modos de trabajo: manual (grabar el recorrido), autónomo (patrullar y misiones) y calibración. El mapa del SLAM y los mapas guardados son comunes a los tres.

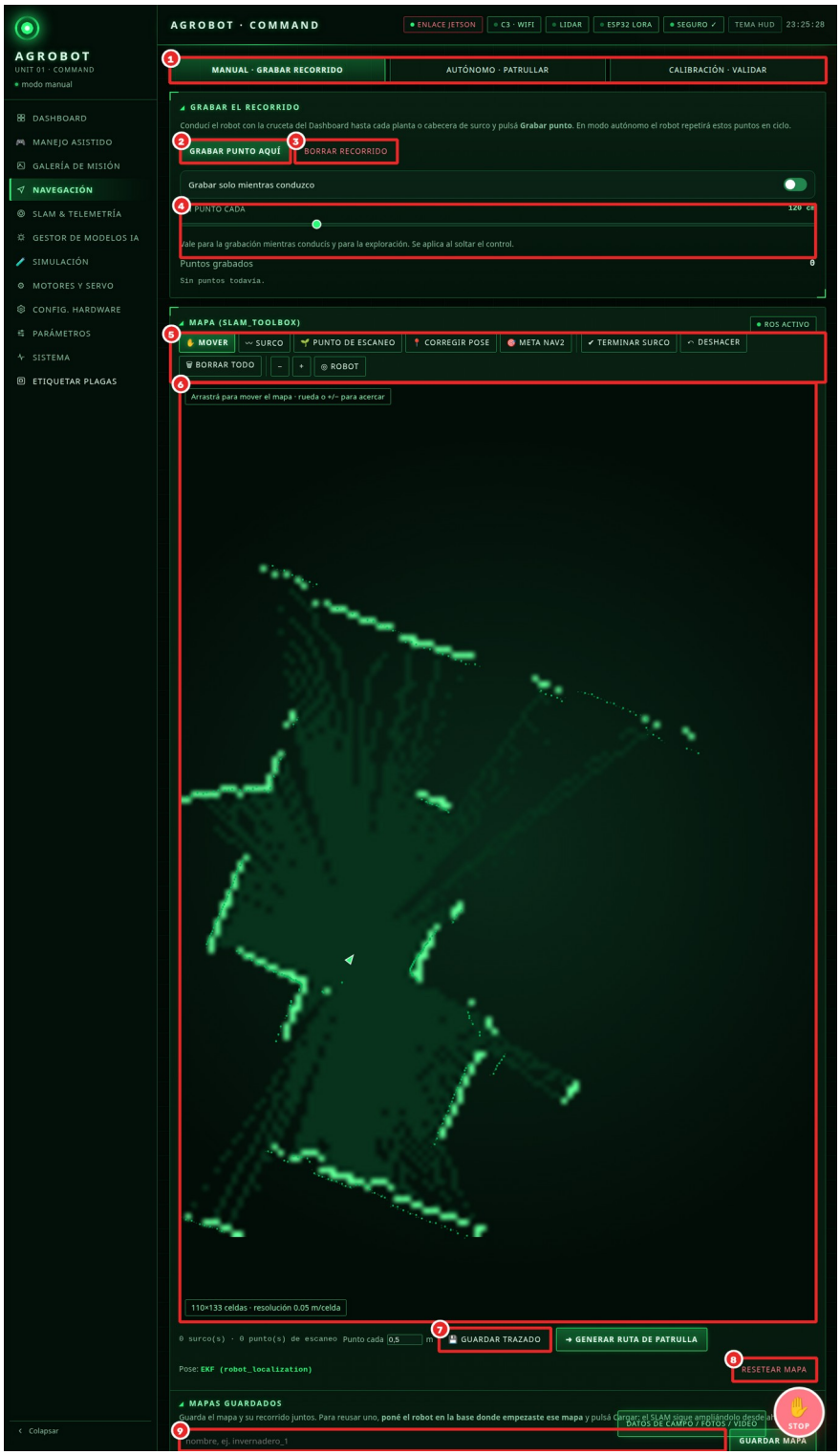


Figura 5. Navegación — modo manual (grabar recorrido y mapa). Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Selector de modo	Manual · grabar recorrido; Autónomo · patrullar; Calibración · validar.	Local	—
2	Grabar punto aquí	Guarda la pose actual como punto del recorrido de patrulla.	POST /api/patrol/record	patrol.py
3	Borrar recorrido	Elimina todos los puntos grabados.	POST /api/patrol/clear	patrol.py
4	Un punto cada…	Distancia entre puntos al grabar mientras se conduce y durante la exploración (10–300 cm).	POST /api/patrol/auto	patrol.py
5	Herramientas del mapa	Mover, trazar surco, punto de escaneo, corregir pose, meta Nav2, terminar surco, deshacer, borrar, zoom y centrar en el robot.	POST /api/training/correct_pose, /api/nav/set_goal	ros_bridge.py, training
6	Mapa SLAM	Mapa de ocupación de slam_toolbox con scan en vivo, robot, surcos, puntos de escaneo y plantas de la misión; arrastrar y zoom con rueda.	WS map, pose, scan_mapa	ros_bridge.py
7	Guardar trazado / generar ruta	Guarda los surcos y puntos y genera la ruta de patrulla con el espaciado elegido.	POST /api/training/save_furrows, save_scan_points, generate_route	jetson_server.py
8	Resetear mapa	Reinicia el mapa del SLAM (pide confirmación).	POST /api/nav/reset_map	ros_bridge.py
9	Guardar mapa	Guarda el mapa del SLAM y su recorrido con un nombre.	POST /api/mapas/guardar	mapas.py
10	Subir datos al simulador	Empaqueta los datos recolectados para el simulador.	POST /api/simulador/subir_datos	subir_datos_simulador.py
11	Mapas guardados	Lista con fecha, puntos y tamaño; CARGAR retoma el SLAM sobre ese mapa; DIGITALIZAR lo convierte en un mundo para el simulador.	GET /api/mapas; POST /api/mapas/cargar|digitalizar	mapas.py, digitalizar_mapa.py

4.5 Navegación — modo autónomo (patrulla, exploración y misiones)

Concentra todas las autonomías del robot. Cada una se inicia y detiene aquí o desde el botón STOP.

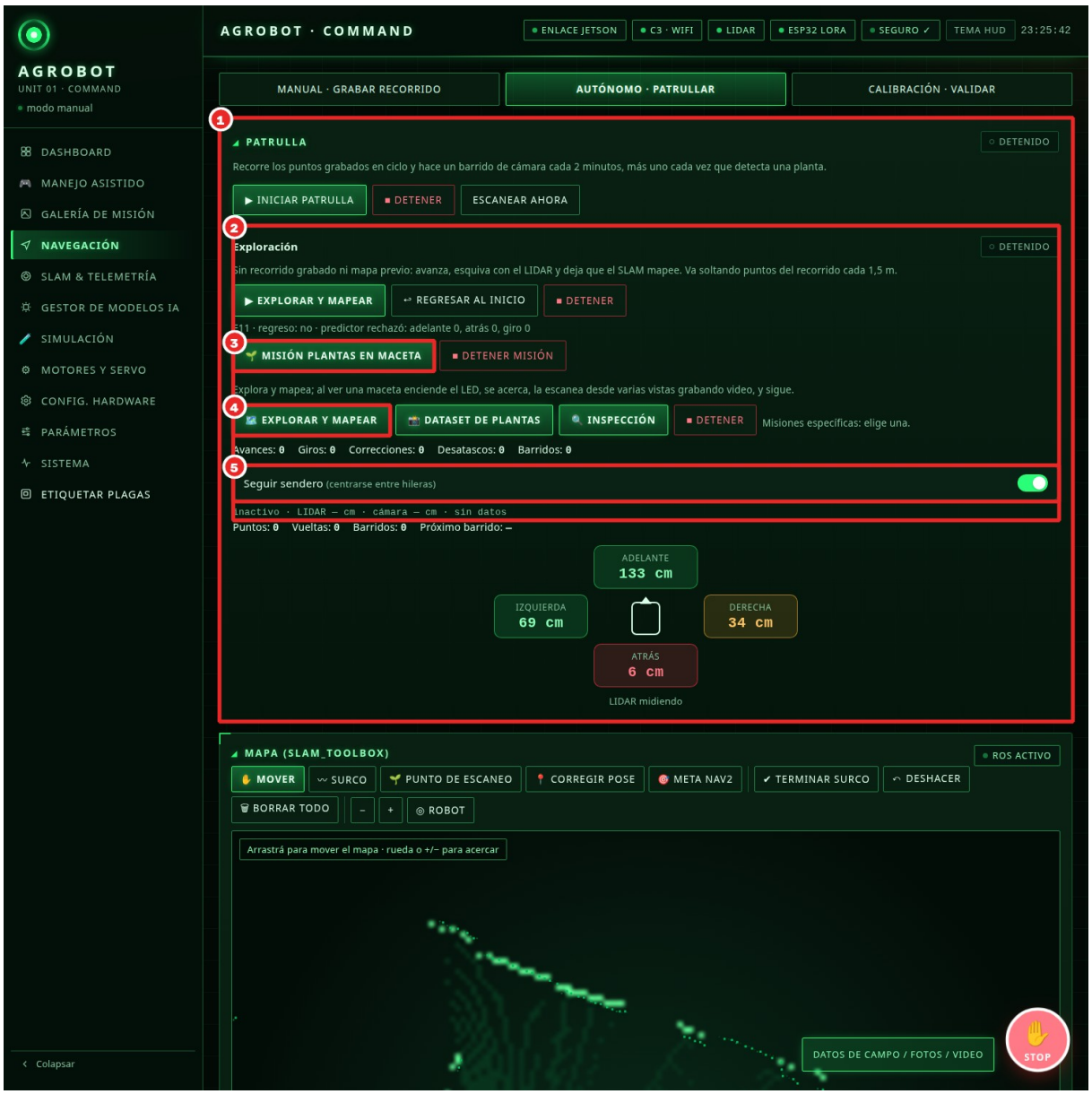


Figura 6. Navegación — modo autónomo (patrulla, exploración y misiones). Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Panel de autonomías	Contiene patrulla, exploración y misiones.	—	—
2	Exploración	EXPLORAR Y MAPEAR avanza, esquiva con el LIDAR y deja que el SLAM mapee; REGRESAR AL INICIO vuelve por la ruta A* de E11; muestra regreso y rechazos del predictor.	POST /api/exploracion/start|stop|regresar	explorador.py, planificador.py
3	Misión plantas en maceta	Explora y mapea; al ver una maceta enciende el LED, se acerca, la escanea desde varias vistas grabando video y sigue.	GET/POST /api/mision_plantas	mision_plantas.py, vision_activa.py
4	Misiones específicas	Explorar y mapear (vuelve al inicio y guarda el mapa), Dataset de plantas (fotos + manifest) e Inspección (veredicto de plaga + informe).	GET/POST /api/misiones	misiones.py
5	Seguir sendero	Se centra entre hileras combinando LIDAR y cámara.	POST /api/params/autonomia	sendero.py

4.6 Navegación — calibración y validación

Graba poses, LIDAR y velocidades mientras se conduce y al terminar valida la calibración de la navegación.

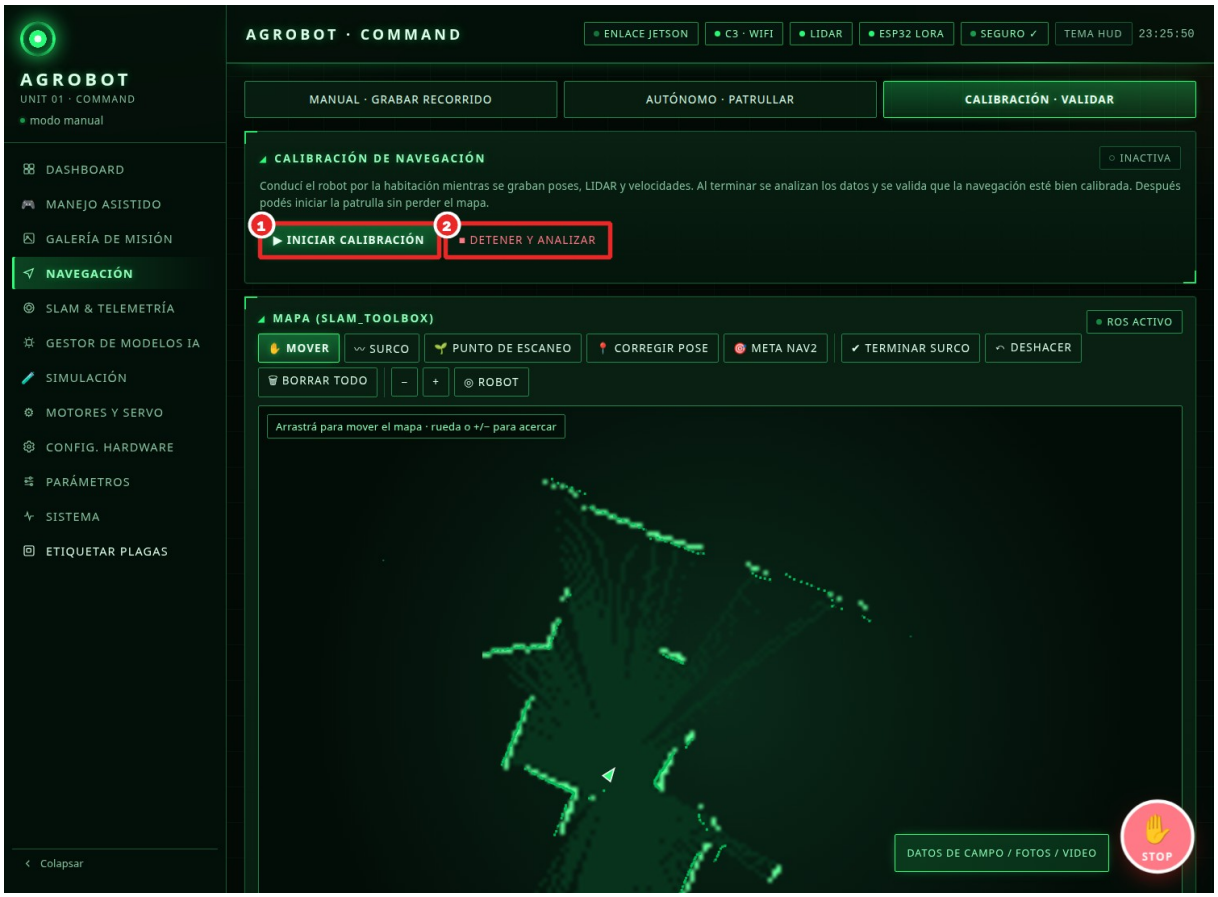


Figura 7. Navegación — calibración y validación. Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Iniciar calibración	Empieza a registrar poses, scans y velocidades.	POST /api/nav_cal/start	navigation_logger.py
2	Detener y analizar	Cierra el registro y analiza si la navegación está bien calibrada.	POST /api/nav_cal/stop; GET /api/nav_cal/status	analizar_navegacion.py

4.7 SLAM y telemetría

Calibración de la odometría visual y vista de toda la telemetría del ESP32 LoRa V3 y del actuador.

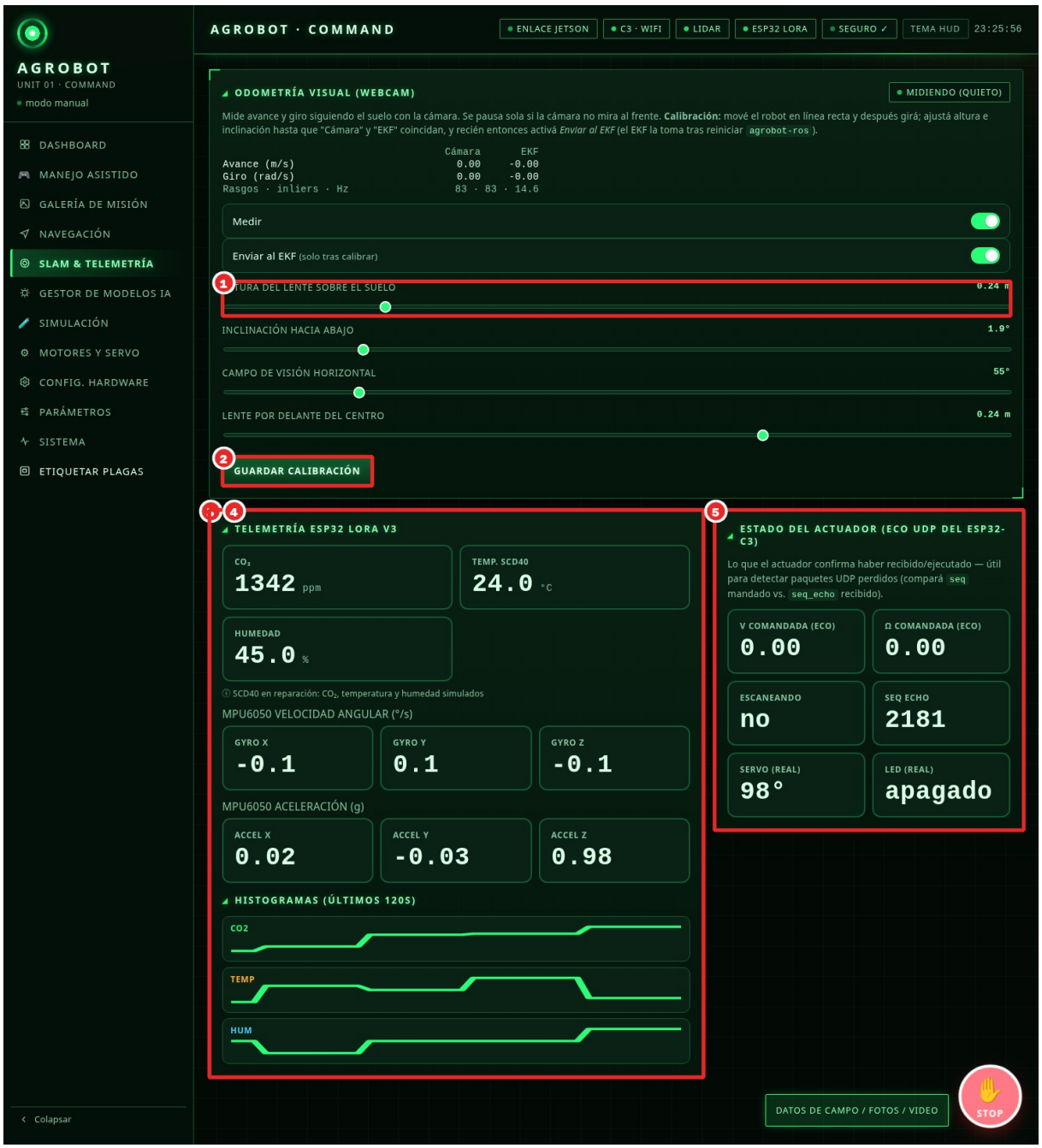


Figura 8. SLAM y telemetría. Los números rojos se explican en la tabla siguiente.

N.°	Control	Qué hace	Canal con el backend	Módulo
1	Calibración de la odometría visual	Altura del lente, inclinación, campo de visión horizontal y posición del lente; deben coincidir «Cámara» y «EKF» antes de enviar al EKF.	POST /api/params/odometria_webcam; WS odom_webcam	odometria_webcam.py
2	Guardar calibración	Guarda la geometría de la cámara (altura 0,24 m, inclinación 1,9°, campo 55° en la última calibración).	POST /api/params/odometria_webcam	config_store.py
3	Telemetría LoRa	CO_2, temperatura SCD40 y humedad, con aviso de que están simulados si el SCD40 no responde; IMU MPU6050 (giroscopio y acelerómetro).	WS env_telemetry	lora_sensor_codec.py, ambiente_simulado.py

N.º	Control	Qué hace	Canal con el backend	Módulo
4	Histogramas	Últimos 120 s de CO_2, temperatura y humedad.	Local (sobre env_telemetry)	—
5	Estado del actuador	Eco UDP del ESP32-C3: v y ω comandadas, escaneo, servo y LED reales, y seq_echo para detectar paquetes perdidos.	WS actuator_status	udp_actuator_link.py

4.8 Gestor de modelos IA

Carga en caliente el modelo de detección activo sin frenar el robot.

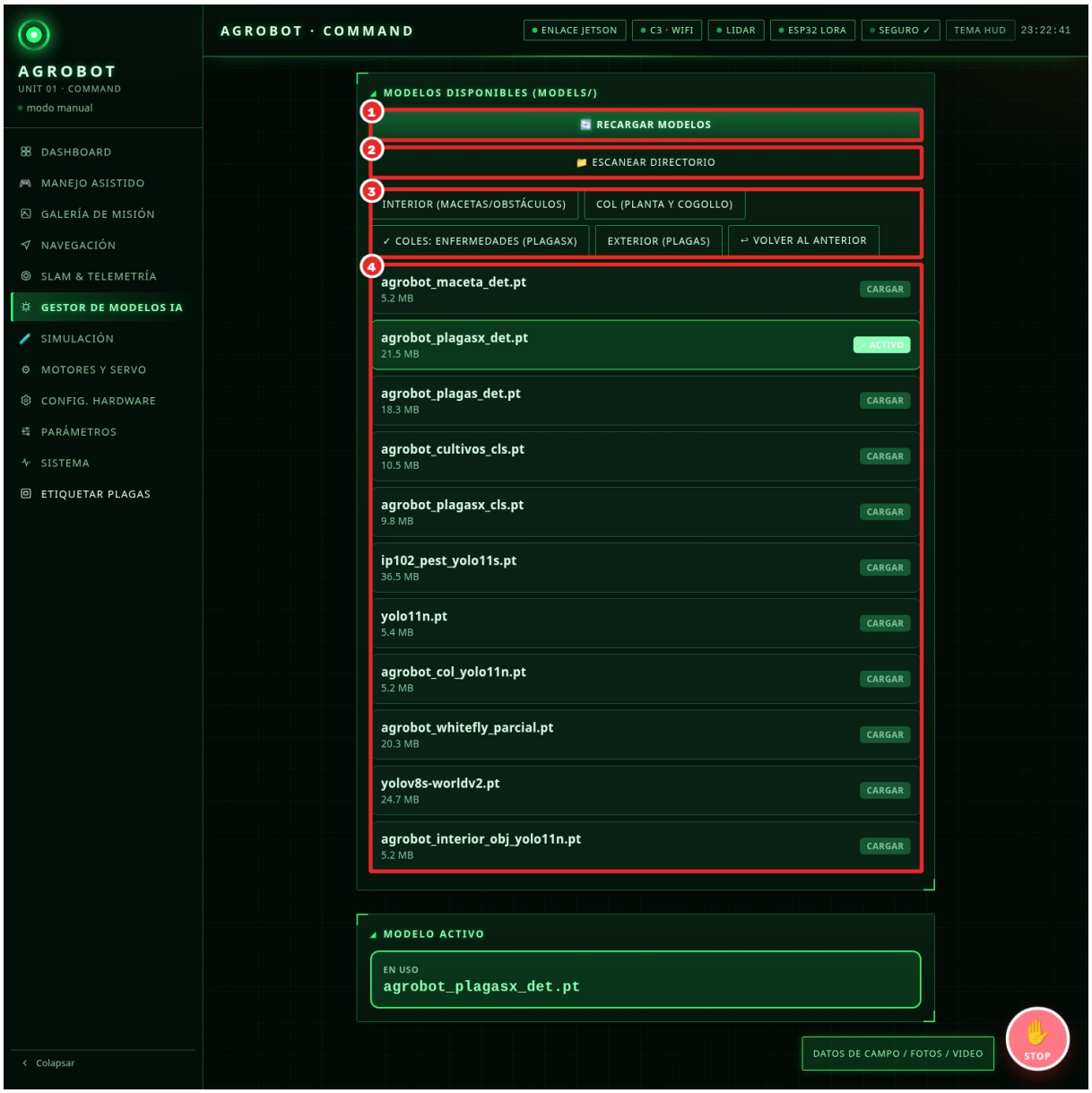


Figura 9. Gestor de modelos IA. Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Recargar / escanear	Vuelve a leer la carpeta models/ y detecta archivos nuevos.	GET /api/models; POST /api/models/scan	modelos_preset.py

N.º	Control	Qué hace	Canal con el backend	Módulo
2	Escanear directorio	Busca modelos nuevos en el disco.	POST /api/models/scan	modelos_preset.py
3	Presets	Interior (macetas/obstáculos), Col (planta y cogollo), Coles: enfermedades (PLAGASX), Exterior (plagas) y Volver al anterior.	GET/POST /api/models/presets	modelos_preset.py
4	Lista de modelos	Cada modelo .pt con tamaño y botón CARGAR; el activo se marca.	POST /api/models/load	camera.py

4.9 Simulación visual

Ejecuta el código real del robot contra un mundo virtual 3D (puerto 8093), embebido en una página aparte.

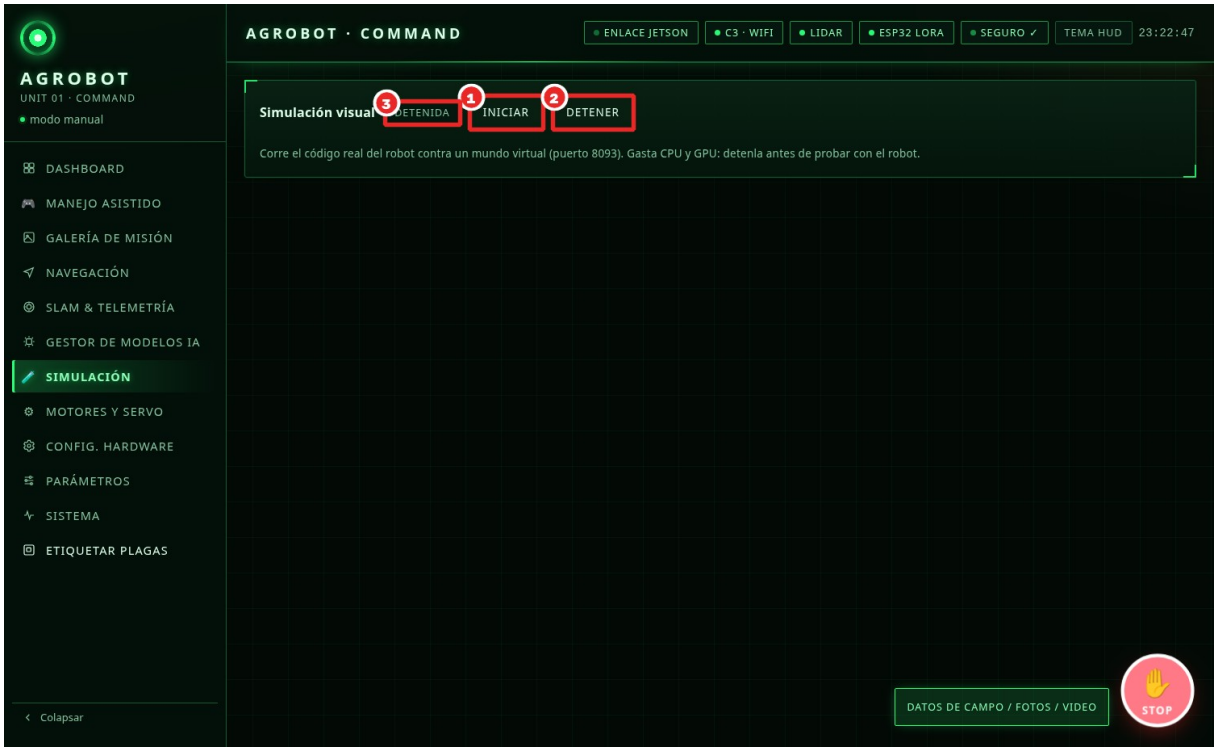


Figura 10. Simulación visual. Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Iniciar	Arranca el simulador visual.	POST /api/simulador	simulador_visual.py
2	Detener	Lo detiene (gasta CPU y GPU: detenerlo antes de probar con el robot).	POST /api/simulador	simulador_visual.py
3	Estado	Detenida / en marcha.	GET /api/simulador	simulador_visual.py

4.10 Motores y servo

Página aparte (`/motores.html`, servida dentro del dashboard) para asignar a qué pin GPIO del ESP32-C3 corresponde cada rueda y probar cada motor.

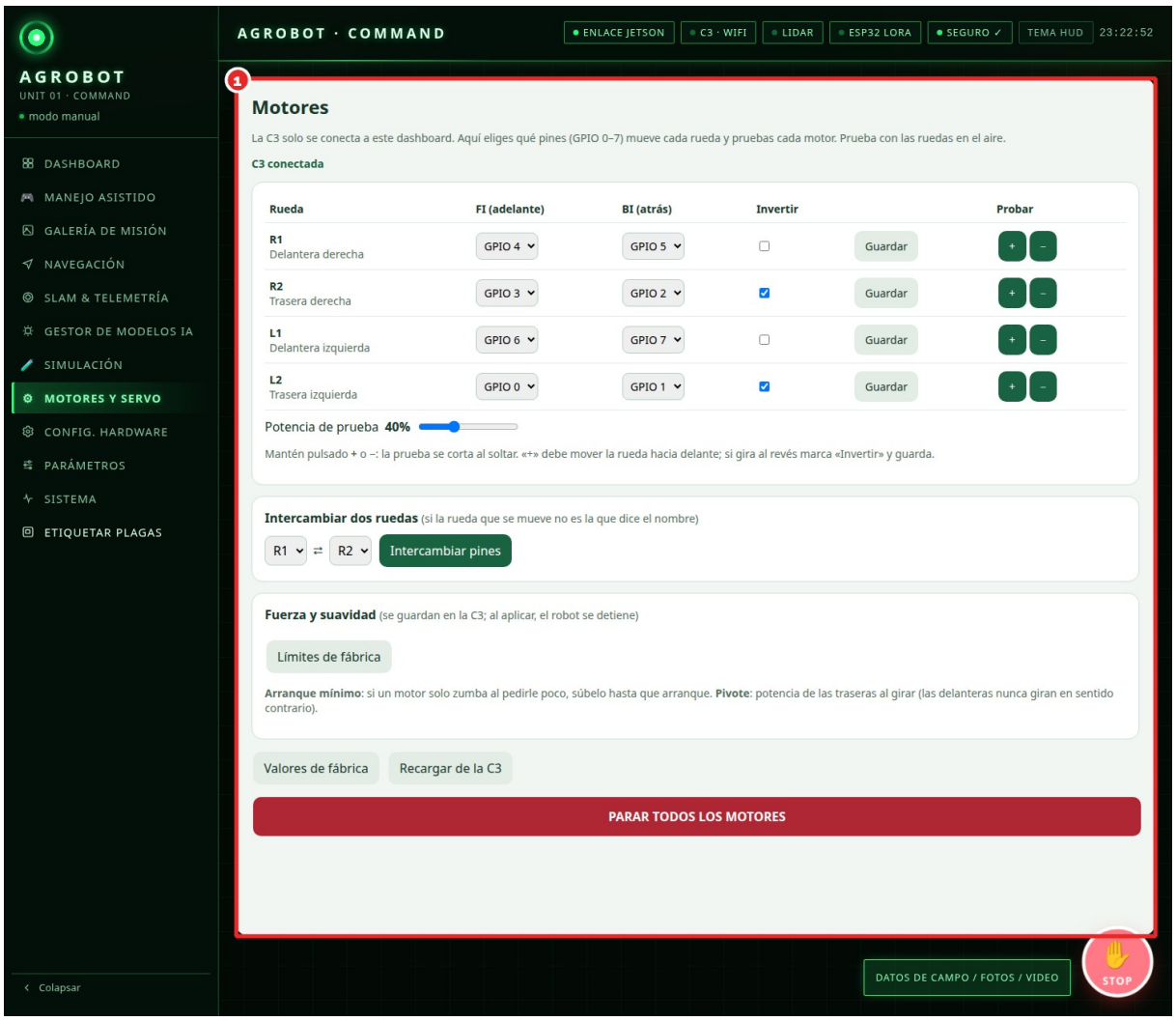


Figura 11. Motores y servo. Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Asignación de motores	Tabla por rueda (R1, R2, L1, L2): pines FI (adelante) y BI (atrás), «Invertir» y botones de prueba por pulsación mantenida; permite intercambiar dos ruedas y ajustar fuerza y suavidad. Prueba siempre con las ruedas en el aire.	Se habla con la C3 por UDP (WiFi), no por el backend	c3_pines.py, c3_setup.py

4.11 Configuración de hardware

Interfaces de la Jetson y calibración mecánica del servo y del giroscopio.

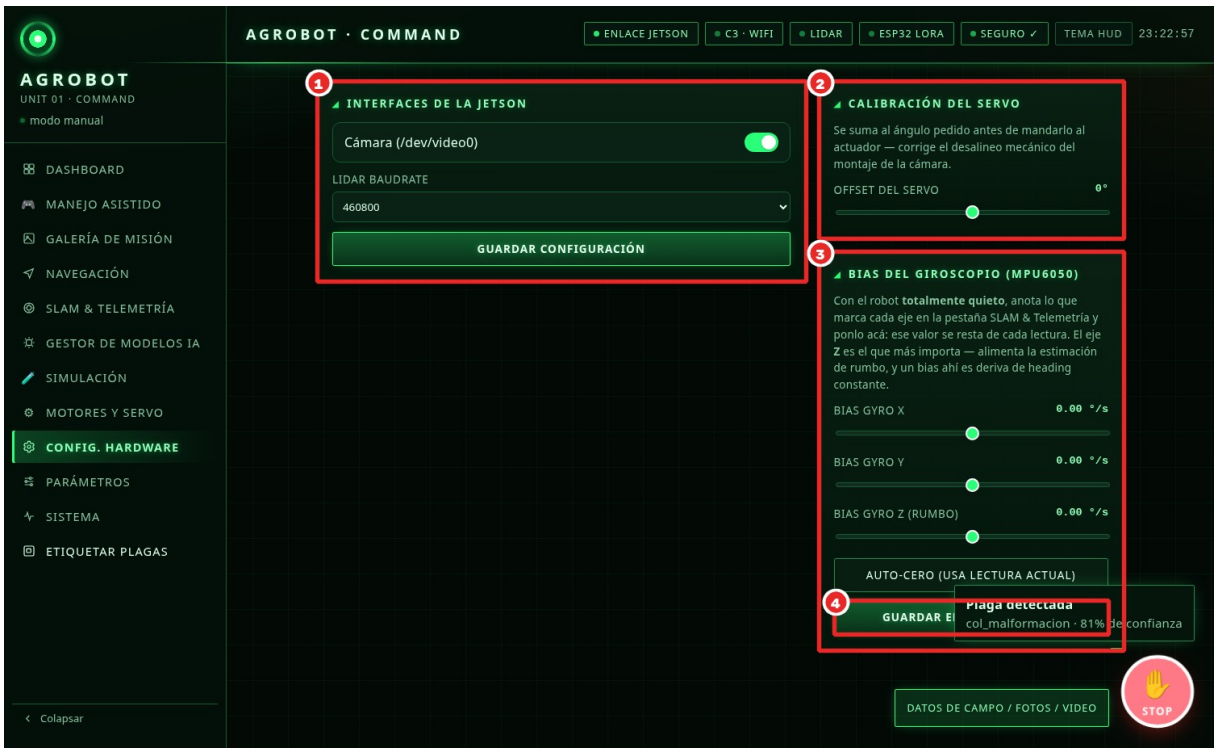


Figura 12. Configuración de hardware. Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Interfaces de la Jetson	Activar la cámara y elegir la velocidad del LIDAR (115200, 230400, 460800).	POST /api/hardware	config_store.py
2	Calibración del servo	Offset en grados que se suma al ángulo pedido: corrige el desalineo mecánico de la cámara.	POST /api/calibration	config_store.py
3	Bias del giroscopio	Con el robot quieto, resta el sesgo de cada eje (el Z es el que más importa: deriva de rumbo). AUTO-CERO usa la lectura actual.	POST /api/calibration	imu_nivel.py, rumbo_fusion.py
4	Guardar en EEPROM local	Persiste los valores en la configuración del backend.	POST /api/calibration	config_store.py

4.12 Parámetros

Todos los parámetros afinables del robot, agrupados en tarjetas; cada una tiene su botón GUARDAR y se persiste en `agrobot_config.json`.

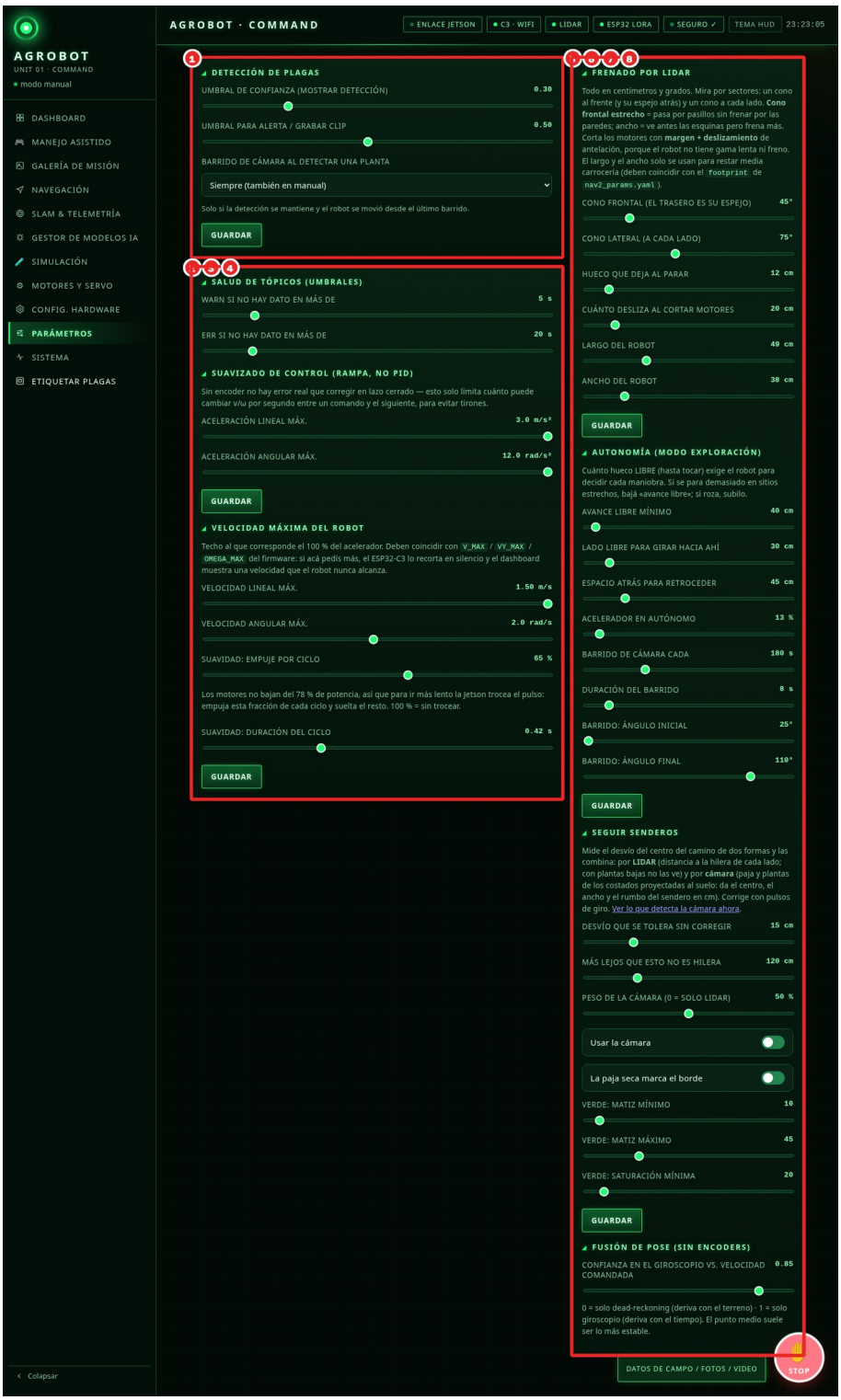


Figura 13. Parámetros. Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Detección de plagas	Umbral de confianza para mostrar, umbral de alerta/grabar clip y cuándo hacer el barrido de cámara al ver una planta.	POST /api/params/detection	camera.py
2	Salud de tópicos	Segundos sin dato para marcar aviso (warn) o error (err) en la pestaña Sistema.	POST /api/params/safety	health.py

N.º	Control	Qué hace	Canal con el backend	Módulo
3	Suavizado de control	Aceleración lineal y angular máximas (rampa, no PID) para evitar tirones.	POST /api/params/control	udp_actuator_link.py
4	Velocidad máxima	Techo del 100 % del acelerador (debe coincidir con el firmware) y suavidad (empuje por ciclo y duración) para ir más lento con motores que no bajan del 78 %.	POST /api/params/control	pulse_drive.py
5	Frenado por LIDAR	Cono frontal (y su espejo atrás), cono lateral, hueco al parar, deslizamiento y medidas del robot.	POST /api/params/slam	anticolision.py , lidar_reactive
6	Autonomía (exploración)	Huecos mínimos para avanzar, girar y retroceder, acelerador autónomo y barrido de cámara (cada cuánto, cuánto dura, ángulos).	POST /api/params/autonomia	explorador.py
7	Seguir senderos	Tolerancia, distancia a pared, peso LIDAR/cámara y rangos HSV del sendero.	POST /api/params/autonomia	sendero.py
8	Fusión de pose	Peso del giroscopio en la estimación de rumbo sin encoders.	POST /api/params/slam	rumbo_fusion .py

4.13 Sistema

Panel de salud: qué componentes están vivos, el radar completo y el consumo de la Jetson.

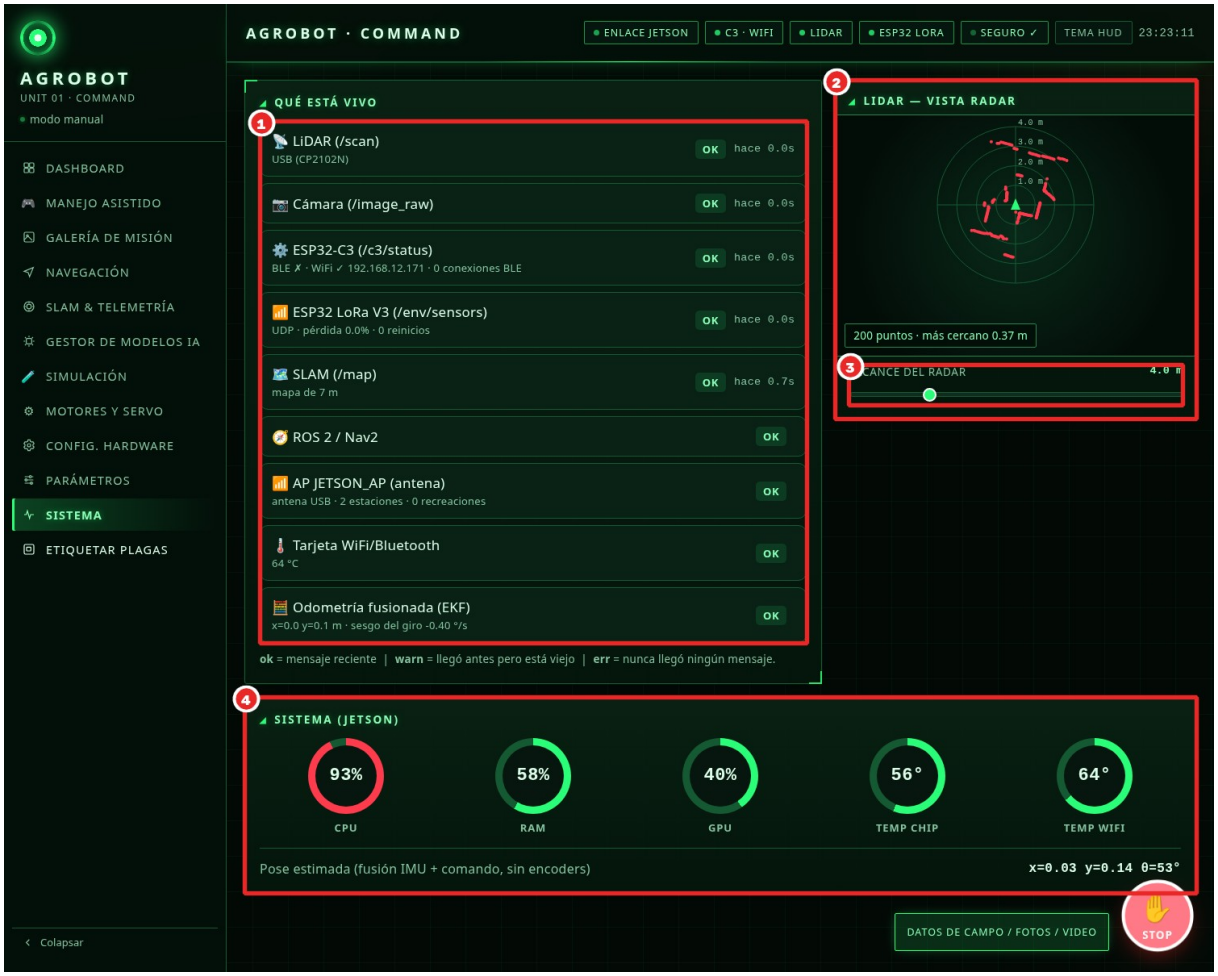


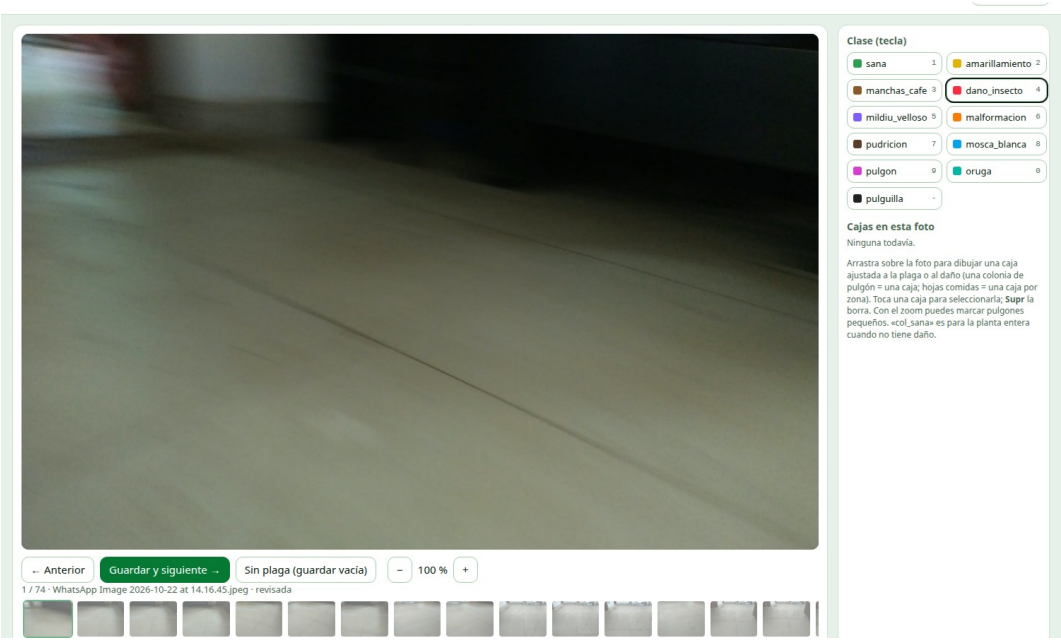
Figura 14. Sistema. Los números rojos se explican en la tabla siguiente.

N.º	Control	Qué hace	Canal con el backend	Módulo
1	Qué está vivo	LIDAR, cámara, ESP32-C3, ESP32 LoRa, SLAM, ROS 2/Nav2, punto de acceso WiFi, temperatura de la tarjeta y odometría fusionada (EKF), cada uno con su edad y estado ok/warn/err.	GET /api/health; WS health	jetson_server.py (_salud_completa)
2	Radar LIDAR	Vista polar a tamaño completo con control de alcance.	WS lidar_scan	lidar_reactive
3	Alcance del radar	0,5–16 m.	Local	—
4	Sistema (Jetson)	CPU, RAM, GPU, temperatura del chip y del módulo WiFi, y la pose estimada.	WS system_stats, pose	health.py

5. Páginas auxiliares

5.1 Etiquetar plagas (/etiquetar.html)

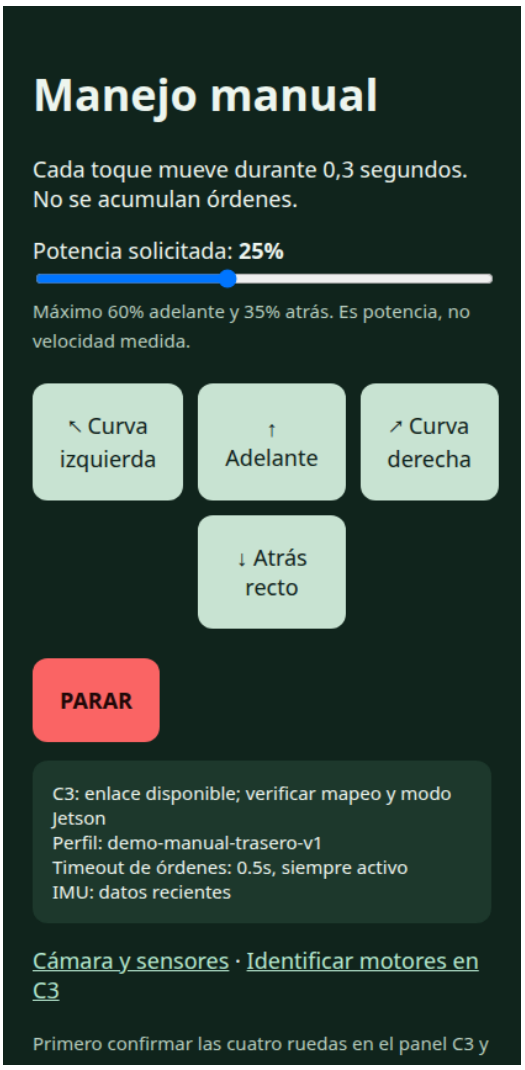
Herramienta aparte para crear el dataset de plagas: se elige la fuente de fotos (mis fotos, capturas del robot, recolección de campo), se dibujan cajas arrastrando sobre la foto, se asigna la clase con el teclado, se guarda y pasa a la siguiente, y se exporta todo en un zip. Rutas: /api/etiquetado/lista, img, etiquetas, resumen y exportar (módulo etiquetado.py).



5.1 Etiquetar plagas (/etiquetar.html).

5.2 Manejo manual para el teléfono (/manejo.html)

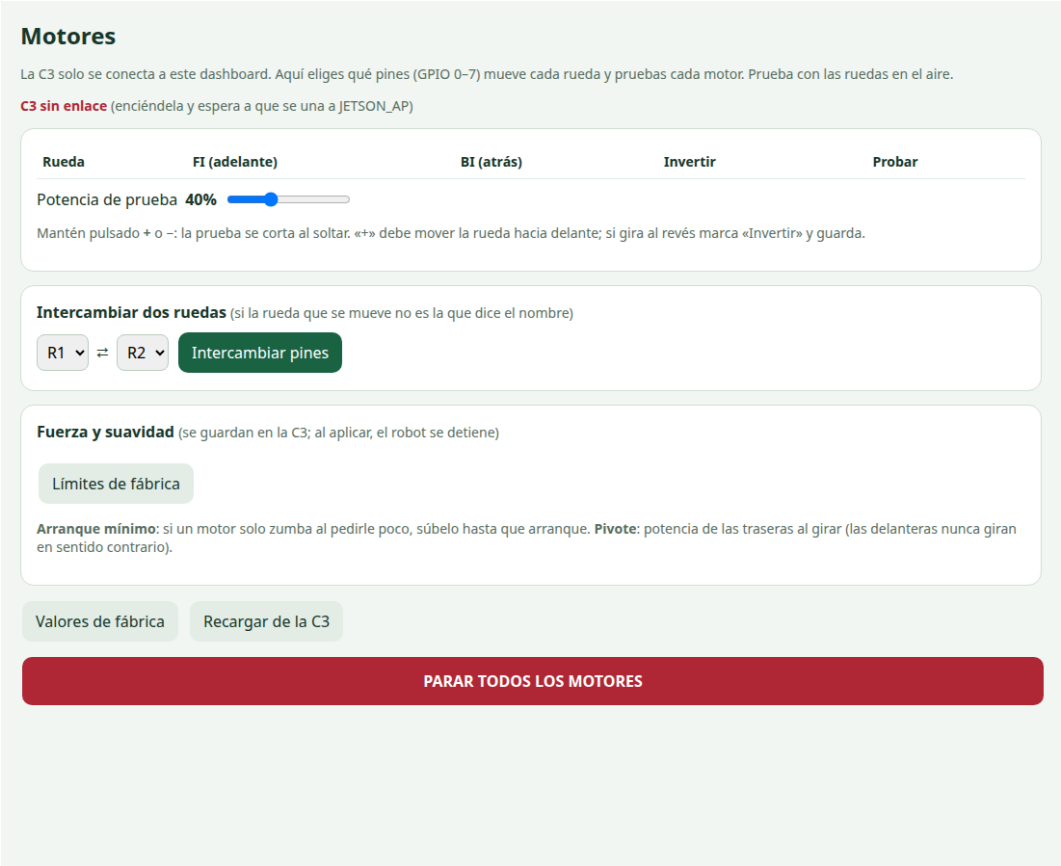
Versión mínima pensada para el móvil: cada toque mueve 0,3 s sin acumular órdenes, potencia limitada (máximo 60 % adelante y 35 % atrás), curvas, parar y comprobación de la conexión. Rutas: /api/drive, /api/emergencia y /api/estado/completo.



5.2 Manejo manual para el teléfono (/manejo.html).

5.3 Motores (/motores.html + motores.js)

Se incrusta en la pestaña «Motores y servo». Asigna a cada rueda su pin FI/BI del ESP32-C3, permite invertir el sentido, probar cada motor con pulsación mantenida e intercambiar dos ruedas; habla directamente con la C3. Dentro del dashboard se ve con tema claro (no sigue el tema HUD).



5.3 Motores (/motores.html + motores.js).

5.4 Widget de redes (widget_redes.html)

Panel «Control de redes»: consulta el estado de la comunicación (GET /api/comunicacion/estado) y cambia el modo WiFi (POST /api/wifi/modo). Está pensado para integrarse en el dashboard; en esta versión no se enlaza desde index.html.

6. Comunicación con el backend

6.1 WebSocket (tiempo real)

Mensaje	Sentido	Lo maneja el dashboard	Contenido
actuator	navegador → backend	—	Servo/LED (navegador → backend)
actuator_setpoint	backend → navegador	sí	Servo y LED pedidos
actuator_status	backend → navegador	sí	Eco del actuador (v, ω, servo, LED, seq)
asistido	backend → navegador	sí	Estado del manejo asistido
capture	backend → navegador	sí	Nueva captura guardada en la galería
captures_cleared	backend → navegador	sí	La galería se vació

Mensaje	Sentido	Lo maneja el dashboard	Contenido
config_update	backend → navegador	sí	La configuración cambió
detection	backend → navegador	sí	Detección de IA (clase, confianza)
drive_mode	backend → navegador	sí	Modo de manejo vigente
emergency_stop	navegador → backend	—	Parada de emergencia (navegador → backend)
entorno	backend → navegador	sí	Sugerencia del clasificador de entorno
env_telemetry	backend → navegador	sí	Telemetría ambiental e IMU del ESP32 LoRa
error	backend → navegador	no	Error del backend
evasion	backend → navegador	no	Estado de la evasión
exploracion	backend → navegador	sí	Estado del explorador (pulsos, giros, estado)
exploracion_event	backend → navegador	sí	Eventos de la exploración
health	backend → navegador	sí	Salud de componentes
lidar_distancias	backend → navegador	sí	Huecos libres adelante/atrás/izquierda/derecha
lidar_scan	backend → navegador	sí	Scan crudo (ángulo, distancia) para el radar
map	backend → navegador	sí	Mapa de ocupación del SLAM
mision_plantas	backend → navegador	no	Eventos de la misión de plantas
misiones	backend → navegador	no	Eventos de las misiones específicas
mission_state	backend → navegador	sí	Misión activa/inactiva
obstacle_warning	backend → navegador	sí	Aviso de frenado por obstáculo
odom_webcam	backend → navegador	sí	Odometría visual (cámara vs EKF)
patrol_event	backend → navegador	no	Eventos de la patrulla
pose	backend → navegador	sí	Pose del robot (SLAM o EKF)
scan_mapa	backend → navegador	sí	Scan del LIDAR en el marco del mapa, para el mapa y el minimapa
seguridad	backend → navegador	sí	Nivel del modo seguro
snapshot	backend → navegador	sí	Estado inicial completo al conectarse
system_stats	backend → navegador	sí	CPU, RAM, GPU y temperaturas de la Jetson
toast	backend → navegador	sí	Aviso emergente para el operador
vision_activa	backend → navegador	no	Estado de la visión activa
zoom_auto	backend → navegador	no	Zoom automático de la cámara

El backend emite 8 tipos de mensaje que el dashboard no trata de forma explícita: emergency_stop, error, evasion, mision_plantas, misiones, patrol_event, vision_activa, zoom_auto. Algunos son solo para trazas o para otros clientes, pero conviene revisarlos: los eventos de misión, patrulla y visión activa llegan al navegador y se descartan en silencio.

6.2 API REST

Grupo (/api/...)	Rutas	Para qué sirve
c3	10	ESP32-C3 (actuador): pines, emparejado, BLE/WiFi, configuración UDP
nav	7	Navegación con Nav2 (meta, reinicio de mapa)
costmap	7	Costmap local
training	6	Trazado de surcos, puntos de escaneo y rutas
etiquetado	6	Herramienta de etiquetado de plagas
mapas	5	Guardar, cargar y digitalizar mapas
patrol	5	Patrulla, recorrido y barridos
camara	4	Zoom, cajas detectadas e imagen cruda
asistido	4	Manejo asistido y aprendizaje
drive	3	Manejo manual por pulsos y modo de manejo
evasion	3	Evasión de obstáculos
exploracion	3	Exploración autónoma y regreso al inicio
simulador	3	Simulador visual
models	3	Modelos de detección y presets
seguridad	3	Modo seguro
captures	3	Capturas de plaga
nav_cal	3	Calibración de la navegación
motor	3	Pruebas de motores
video	2	Flujo MJPEG y grabación de video
paso	2	Paso robótico (avance medido por pasos)
suelo	2	Suelo libre visto por la cámara
giro_cerrado	2	Giro de lazo cerrado con giroscopio
maniobra_giro	2	Maniobra de giro 180°/360°
misiones	2	Misiones específicas (mapear, dataset, inspección)
mision_plantas	2	Misión de plantas en maceta
vision_activa	2	Visión activa (acercarse y escanear)
hardware	2	Interfaces de hardware
calibration	2	Calibración de servo y giroscopio
mission	2	Inicio/parada de la autonomía
entorno	2	Clasificador de entorno
params	2	Parámetros persistentes
raíz	2	Páginas y archivos estáticos
recoleccion	2	Recolección de campo (fotos, video, muestras)
ws	1	WebSocket de tiempo real
emergencia	1	Parada de emergencia
sendero	1	Vista del detector de senderos
saludo	1	Saludo de demostración
session-report	1	Reporte de sesión
asistente	1	Asistente
calibracion	1	Calibración de la cámara
lidar	1	Estado del LIDAR
obstacle_status	1	Estado de la anticolisión
ocupado	1	¿Robot ocupado? (consulta del vigía del reloj)
health	1	Salud de componentes
training.html	1	Página heredada
comunicacion	1	Calidad de la comunicación
wifi	1	Modo WiFi
diagnostico	1	Diagnóstico

Grupo (/api/...)	Rutas	Para qué sirve
estado	1	Estado completo
recoleccion-export	1	

En total hay 128 rutas registradas (54 GET y 72 POST); 72 las usa alguna página del dashboard y el resto sirve a scripts, herramientas y a la integración con otros módulos. La lista completa está en el Anexo A.

7. Seguridad y robustez

Mecanismos verificados en el código:

- Parada de emergencia redundante: el botón STOP envía el mensaje `emergency_stop` por WebSocket y una petición POST a /api/emergencia con `keepalive`, y actúa en el primer toque (pointerdown), no al soltar. Detiene misión, exploración, patrulla, Nav2, pulsos y motores en un único sitio del backend (_parada_total).
- Reconexión automática: si el WebSocket se cierra, el indicador «enlace Jetson» se pone en rojo y se reintenta cada segundo.
- Confirmaciones: 11 diálogos de confirmación antes de acciones destructivas o que mueven el robot (vaciar la galería, resetear el mapa, iniciar misiones...).
- Acceso: HTTP Basic con clave y cookie de sesión (el WebSocket usa la cookie) cuando existe el archivo de clave del dashboard; las peticiones desde la propia Jetson (127.0.0.1) no la necesitan. Sin ese archivo el dashboard queda abierto, pensado para la red aislada del punto de acceso. En esta instalación el archivo existe.
- Perfil de demostración: con `control.solo_manual` activo, el backend rechaza (409) el inicio de misión, exploración, evasión, patrulla automática, metas de Nav2, paso robótico y calibración de navegación.
- Modo seguro: el indicador «Seguridad» muestra su nivel y permite rearmarlo (POST /api/seguridad/rearmar); lo activa el backend ante condiciones peligrosas como un vuelco.
- Tolerancia a fallos del navegador: localStorage siempre dentro de try/catch (modo privado o datos bloqueados) y la interfaz funciona sin él; las preferencias (tema, acelerador, alcance del minimapa) se recuerdan por navegador.
- Diseño pensado para el campo: tema claro para leer la tablet a pleno sol, tema HUD para poca luz, controles grandes y atajos de teclado.

Diseño visual: tema HUD y tema claro

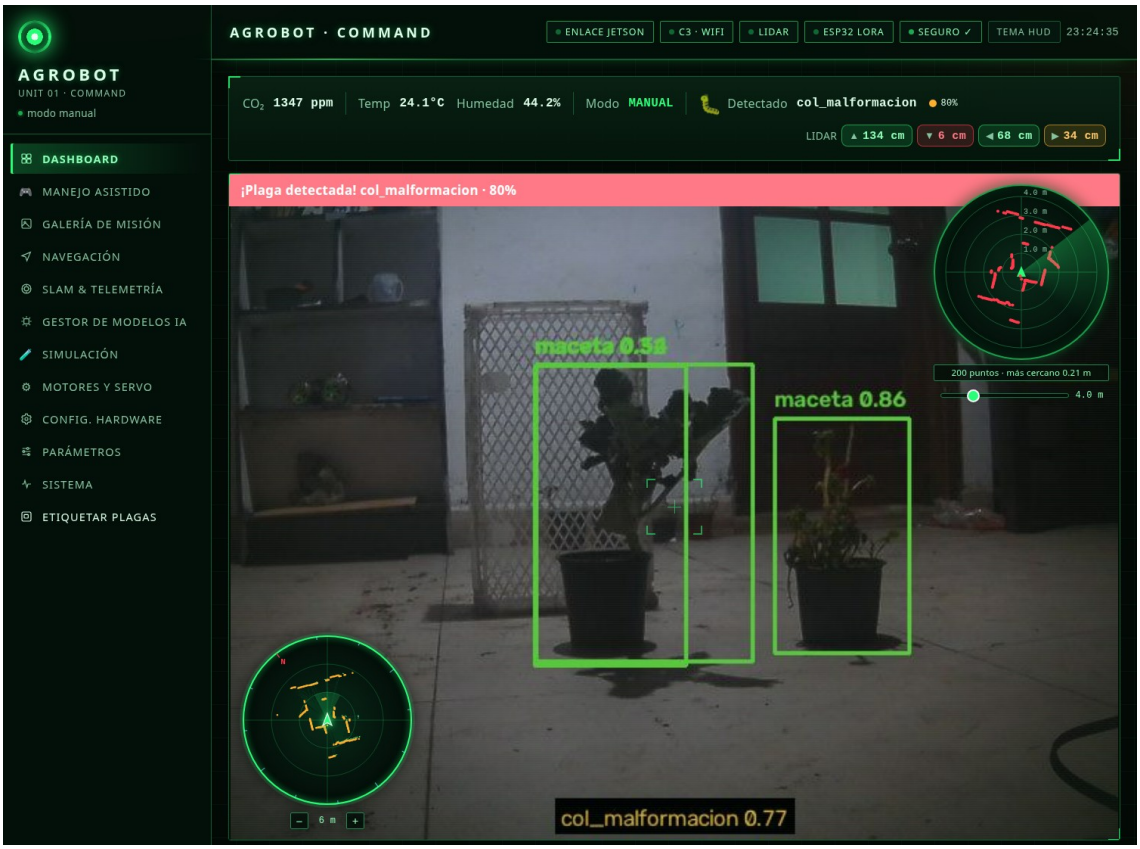


Figura 15. Tema HUD (verde sobre fondo oscuro), el predeterminado.

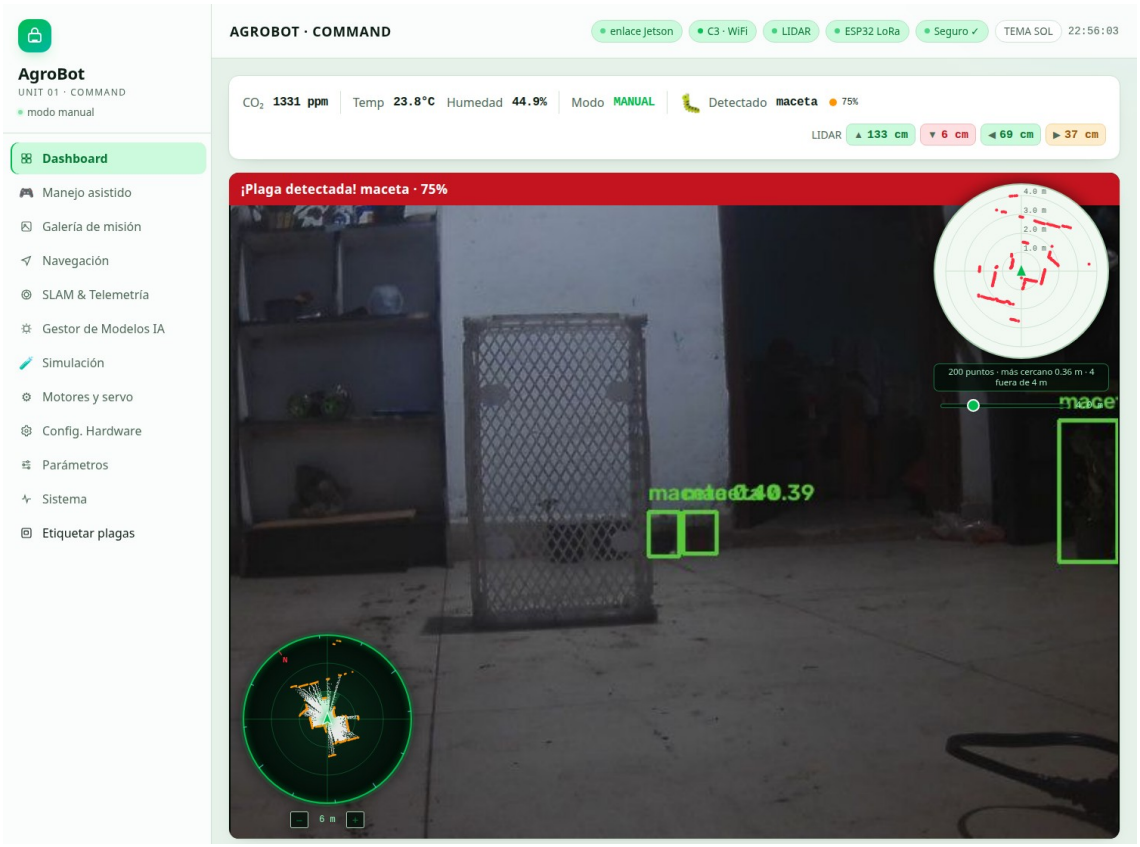


Figura 16. Tema claro («SOL») para leer a pleno sol; se alterna con el botón «TEMA» de la barra superior.

8. Evaluación crítica

Fortalezas

- Una sola pantalla da el estado completo (video, mapa, LIDAR, telemetría, detecciones) y el control: no hace falta abrir herramientas externas para operar.
- Funciona sin internet y sin instalar nada: abrir una URL basta, desde PC, tablet o móvil.
- Cada control está conectado a una ruta o mensaje concreto del backend, con módulos separados por responsabilidad; la configuración se persiste y se puede afinar sin tocar código (pestaña Parámetros).
- Seguridad pensada: STOP redundante, confirmaciones, perfil de demostración y modo seguro.

Limitaciones y mejoras posibles

- Archivo monolítico: 4259 líneas de HTML, CSS y JavaScript en un solo archivo, sin módulos ni pruebas automáticas del frontend; cada cambio se verifica a mano o con capturas.
- 8 tipos de mensaje WebSocket emitidos no se manejan en el navegador (emergency_stop, error, evasion, mision_plantas, misiones, patrol_event, vision_activa, zoom_auto).
- 56 de las 128 rutas REST no las usa ninguna página: conviene documentarlas o retirar las muertas.
- Accesibilidad limitada: 6 etiquetas aria-label y 32 descripciones emergentes (title) para 93 botones; muchos botones se entienden solo por su icono.
- Textos con mezcla de voseo y tuteo («tocá», «poné», «tú») y sin soporte de otros idiomas.
- La clave del dashboard es opcional: sin el archivo de clave cualquier equipo de la red del punto de acceso puede mover el robot.
- Las pestañas Simulación y Motores son páginas incrustadas con su propio estilo (Motores es claro y no sigue el tema HUD).
- Misiones, maniobra de giro 180°/360° y vigía del reloj se añadieron el 27-oct y siguen sin probarse en el robot real.

Problemas encontrados y corregidos durante este análisis

Problema	Evidencia	Corrección
Slider del LIDAR desaparecía en pantallas de menos de 900 px	Revisión visual en Manejo asistido a 820 px	Se mantiene visible junto al radar
Título «AGROBOT» tapado por los indicadores a anchos medios	Captura a 820 px	Los indicadores pasan a una segunda fila y el título se acorta
Lista de pestañas sin forma de desplazarse en ventanas bajas	Petición del operador	Scroll con deslizador verde siempre visible
Banner «¡Plaga detectada!» saltaba con la clase «maceta»	Captura del dashboard en vivo	Las clases de planta se ignoran en el banner
Cajas de plaga que ocupan media pantalla rodeaban el video	Fotos de la prueba de exterior (4 de 4 cajas >60 % del cuadro) y una captura del ~40 %	Las cajas de plaga de ≥30 % del cuadro se anotan como rótulo centrado abajo

9. Conclusiones

El dashboard cubre de punta a punta el ciclo de operación del robot: monitoreo en tiempo real, manejo manual y asistido, mapeo y navegación, autonomías y misiones, gestión de modelos de IA, calibración y parámetros, salud del sistema y recolección/etiquetado de datos para entrenar. Su principal fortaleza es que todo está en una sola pantalla y se conecta de forma directa con módulos concretos del backend; sus principales debilidades son de mantenibilidad (archivo monolítico, sin pruebas) y de cobertura (mensajes sin tratar, rutas sin usar, accesibilidad). Las mejoras más rentables serían separar el JavaScript en módulos con pruebas, tratar los eventos de misión y patrulla en la interfaz, y exigir clave por defecto.

Anexo A. Rutas REST registradas

Método	Ruta	Usada por	Descripción (primera frase del docstring)
GET	/	—	
STATIC	/	—	
GET	/api/asistente/estado	—	GET /api/asistente/estado — foto compacta de TODO lo que hace falta para manejar el robot de forma asistida (C
GET	/api/asistido	index	GET /api/asistido — estado del mando, resumen de correcciones y ajustes sugeridos.
POST	/api/asistido/aplicar	index	POST /api/asistido/aplicar {"id": ...} — aplica UNA sugerencia (nunca se aplican solas).
POST	/api/asistido/reanudar	index	POST /api/asistido/reanudar — devuelve el mando a la autonomía que se interrumpió.
POST	/api/asistido/soltar	index	POST /api/asistido/soltar — se queda en manual: la autonomía interrumpida NO se reanuda.
GET	/api/c3/limites	—	
POST	/api/c3/limites	—	
POST	/api/c3/limites/reset	—	
GET	/api/c3/pines	—	
POST	/api/c3/pines	—	
POST	/api/c3/pines/intercambiar	—	
POST	/api/c3/pines/reset	—	
POST	/api/c3/prueba	—	
GET	/api/c3/setup	—	
POST	/api/c3/setup/{action}	—	
POST	/api/calibracion/imu_nivel	—	POST /api/calibracion/imu_nivel {"reset": true} para borrarla. Con el robot QUIETO y NIVELADO: promedia ~5 s d
GET	/api/calibration	index	
POST	/api/calibration	index	
GET	/api/camara/cajas	—	GET /api/camara/cajas — lo que el robot detecta AHORA (clase, confianza, caja y distancia por cámara a la cara
GET	/api/camara/cruda.jpg	—	GET /api/camara/cruda.jpg — el último fotograma SIN cajas dibujadas.
GET	/api/camara/zoom	index	GET/POST /api/camara/zoom {"zoom": 1..4} — zoom de la cámara para inspeccionar plantas/plagas. Con zoom > 1 se
POST	/api/camara/zoom	index	GET/POST /api/camara/zoom {"zoom": 1..4} — zoom de la cámara para inspeccionar plantas/plagas. Con zoom > 1 se
GET	/api/captures	index	
POST	/api/captures/clear	index	
GET	/api/comunicacion/estado	—	Retorna estado de todos los sistemas de comunicación.
GET	/api/costmap	—	GET /api/costmap — resumen del costmap local.
POST	/api/costmap/delete	—	POST /api/costmap/delete — borra un mapa guardado.

Método	Ruta	Usada por	Descripción (primera frase del docstring)
GET	/api/costmap/grid	—	GET /api/costmap/grid — rejilla como lista de listas (para debug/visor).
POST	/api/costmap/load	—	POST /api/costmap/load — restaura un costmap guardado.
GET	/api/costmap/maps	—	GET /api/costmap/maps — lista de mapas locales guardados.
POST	/api/costmap/reset	—	POST /api/costmap/reset
POST	/api/costmap/save	—	POST /api/costmap/save — guarda el costmap local con nombre.
GET	/api/diagnostico/wifi	—	Retorna análisis detallado del WiFi (caídas, salud, etc).
POST	/api/drive	index, manejo	POST /api/drive {"accion": "adelante|atras|izquierda|derecha|parar", "escala": 0.1..1, "maniobra": true}
GET	/api/drive/mode	index	GET: modo actual. POST {"modo": ..., "perfil": bool}: cambia el modo.
POST	/api/drive/mode	index	GET: modo actual. POST {"modo": ..., "perfil": bool}: cambia el modo.
POST	/api/emergencia	index, manejo	POST /api/emergencia — parada total por HTTP (el WebSocket se cae al bloquear la pantalla del móvil; una petic
GET	/api/entorno	index	GET /api/entorno — qué cree el robot que es el entorno. POST {"activo": true|false} lo activa/apaga (persisten
POST	/api/entorno	index	GET /api/entorno — qué cree el robot que es el entorno. POST {"activo": true|false} lo activa/apaga (persisten
GET	/api/estado/completo	manejo	
GET	/api/etiquetado/etiquetas/{id}	etiquetar	
POST	/api/etiquetado/etiquetas/{id}	etiquetar	
GET	/api/etiquetado/exportar	etiquetar	Zip listo para subir como dataset de Kaggle: images/, labels/, dataset.yaml (sin particiones: el notebook las
GET	/api/etiquetado/img/{id}	etiquetar	
GET	/api/etiquetado/lista	etiquetar	
GET	/api/etiquetado/resumen	etiquetar	
GET	/api/evasion	—	
POST	/api/evasion/start	—	POST /api/evasion/start {"segundos": 120, "objetivo_deg": 0}
POST	/api/evasion/stop	—	
POST	/api/exploracion/regresar	index	POST /api/exploracion/regresar — E11: el robot da la vuelta y vuelve al punto de salida por una ruta A* sobre
POST	/api/exploracion/start	index	POST /api/exploracion/start — autonomía reactiva: recorre y mapea.
POST	/api/exploracion/stop	index	
GET	/api/giro_cerrado	—	GET /api/giro_cerrado — último giro de lazo cerrado. POST {"grados": 15, "w": 0.6} — gira esos grados (+ izqui
POST	/api/giro_cerrado	—	GET /api/giro_cerrado — último giro de lazo cerrado. POST {"grados": 15, "w": 0.6} — gira esos grados (+ izqui
GET	/api/hardware	index	
POST	/api/hardware	index	
GET	/api/health	—	
POST	/api/lidar/scan	—	Punto de enganche para quien lea el RPLIDAR C1 (por ejemplo un script con la librería rplidar corriendo apar
GET	/api/maniobra_giro	—	GET /api/maniobra_giro — estado. POST {"grados": 180|-180|360|-360, "tramo": 30} — gira ese ángulo en el sitio
POST	/api/maniobra_giro	—	GET /api/maniobra_giro — estado. POST {"grados": 180|-180|360|-360, "tramo": 30} — gira ese ángulo en el sitio
GET	/api/mapas	index	
POST	/api/mapas/cargar	index	POST /api/mapas/cargar {"nombre": "invernadero"}
GET	/api/mapas/digital/{nombre}/{archivo}	—	GET /api/mapas/digital/<nombre>/<archivo>: descarga lo generado por /api/mapas/digitalizar.
POST	/api/mapas/digitalizar	index	POST /api/mapas/digitalizar {"nombre": "invernadero"} -> plano vectorial (SVG, DXF, JSON y PNG) en maps/<nombr
POST	/api/mapas/guardar	index	POST /api/mapas/guardar {"nombre": "invernadero", "sobrescribir": false}
GET	/api/mision_plantas	index	GET/POST /api/mision_plantas {"activa": true|false} — la misión de la demo: explorar, mapear y escanear planta
POST	/api/mision_plantas	index	GET/POST /api/mision_plantas {"activa": true|false} — la misión de la demo: explorar, mapear y escanear planta

Método	Ruta	Usada por	Descripción (primera frase del docstring)
GET	/api/misiones	index	GET /api/misiones — catálogo y estado. POST {"mision": "mapear"|"dataset"|"inspeccion", "accion": "iniciar"|"d
POST	/api/misiones	index	GET /api/misiones — catálogo y estado. POST {"mision": "mapear"|"dataset"|"inspeccion", "accion": "iniciar"|"d
POST	/api/mission/start	index	MODO AUTÓNOMO. Único punto de entrada: arranca la patrulla por los waypoints grabados y carga el YOLO.
POST	/api/mission/stop	index	
GET	/api/models	index	
POST	/api/models/load	index	
GET	/api/models/presets	index	
POST	/api/motor/comando	—	
POST	/api/motor/parar	—	
POST	/api/motor/toggle-safeguard	—	
POST	/api/nav/cancel	—	
GET	/api/nav/mode	—	GET: modo actual. POST {"mode": "mapping"|"localization"}: solo lo anota para la UI — relanzar los nodos ROS c
POST	/api/nav/mode	—	GET: modo actual. POST {"mode": "mapping"|"localization"}: solo lo anota para la UI — relanzar los nodos ROS c
POST	/api/nav/reset_map	index	POST /api/nav/reset_map — borra el mapa de slam_toolbox y empieza de cero.
POST	/api/nav/restart	—	Reinicia el servicio Nav2 (killea y relanza)
POST	/api/nav/save_map	—	POST /api/nav/save_map — cierra la sesión de mapeo: le pide a nav2_map_server que escriba maps/agrobot_map.{ya
POST	/api/nav/set_goal	index	POST /api/nav/set_goal {"x": .., "y": .., "theta_deg": ..} El click en el mini-mapa del dashboard llega aquí y
POST	/api/nav_cal/start	index	
GET	/api/nav_cal/status	index	
POST	/api/nav_cal/stop	index	
GET	/api/obstacle_status	—	Retorna el estado actual del sistema de evasión de obstáculos.
GET	/api/ocupado	—	GET /api/ocupado — {"ocupado": bool, "motivos": [...]}.
GET	/api/params	index	
POST	/api/params/{section}	index	
GET	/api/paso	—	GET /api/paso — en qué va el movimiento robótico por pulsos.
POST	/api/paso	—	POST /api/paso {"accion": "avanzar", "cm": 50} | {"accion": "girar", "grados": -90} | {"accion": "parar"}; opc
GET	/api/patrol	index	
POST	/api/patrol/auto	index	POST /api/patrol/auto {"activa": true} — graba un punto cada 1,5 m mientras el operador conduce, para no tener
POST	/api/patrol/clear	index	
POST	/api/patrol/record	index	Graba el punto donde está el robot ahora mismo.
POST	/api/patrol/scan	index	Barrido manual bajo demanda, sin esperar a los 2 minutos.
GET	/api/recoleccion	index	
GET	/api/recoleccion-export	index	
POST	/api/recoleccion/{action}	index	
POST	/api/saludo	index	POST /api/saludo — izquierda, derecha, mira arriba, foco x3 y barrido de cámara.
GET	/api/seguridad	index	
POST	/api/seguridad/modo-seguro	—	Activa/desactiva el modo seguro completamente.
POST	/api/seguridad/rearmar	index	
GET	/api/sendero/vista.jpg	index	GET /api/sendero/vista.jpg — lo que ve el detector de hileras ahora: cámara con las plantas marcadas + vista c
POST	/api/session-report	index	
GET	/api/simulador	index	GET /api/simulador — estado del simulador visual. POST {"accion": "iniciar"|"detener", "escenario"?}.
POST	/api/simulador	index	GET /api/simulador — estado del simulador visual. POST {"accion": "iniciar"|"detener", "escenario"?}.

Método	Ruta	Usada por	Descripción (primera frase del docstring)
POST	/api/simulador/subir_datos	index	POST /api/simulador/subir_datos {"nombre"?: "...", "mapa"?: "<mapa ya guardado>"}
GET	/api/suelo	—	GET /api/suelo — resumen del suelo libre (sin la máscara).
GET	/api/suelo/vista.jpg	—	GET /api/suelo/vista.jpg — suelo libre por cámara: verde = suelo, rojo = base de obstáculos, recuadro = franja
POST	/api/training/correct_pose	index	
GET	/api/training/data	index	
POST	/api/training/generate_route	index	
POST	/api/training/log_correction	—	
POST	/api/training/save_furrows	index	
POST	/api/training/save_scan_points	index	
POST	/api/video/record	index	Inicia o detiene la grabación de video desde la cámara. POST {"action": "start"|"stop"|"status"}.
GET	/api/vision_activa	—	GET /api/vision_activa — estado de la visión activa y su configuración.
POST	/api/vision_activa	—	POST /api/vision_activa {"activa": true|false} — la ENCIENDE (mueve el servo y el robot en pulsos cortos) o la
POST	/api/wifi/modo	index	Cambia entre 'estacion' (Lora_Wan) y 'patrullaje' (AP propio).
STATIC	/captures	—	
GET	/training.html	—	
GET	/video	index	Sirve el MJPEG. No toca la cámara: solo consume lo que el hilo de captura ya dejó listo (ver camera.py). Antes
GET	/ws	—	

Anexo B. Reproducibilidad

- tools/inventario_dashboard.py — inventario de rutas, mensajes y estadísticas (docs/INFORME_DASHBOARD/inventario.json).
- tools/dashboard_spec.py — especificación de los controles marcados y su descripción.
- tools/capturas_anotadas_dashboard.py — capturas con números rojos (requiere el backend en marcha).
- tools/generar_informe_dashboard.py — este documento.